

DeepTriangle: An Actuarial Guide to Kevin Kuo's Deep Learning Reserving Model

Kevin Kuo's **DeepTriangle** is best understood as an attempt to modernise aggregate loss reserving without abandoning the development triangle itself.

Traditional chain ladder asks a fairly constrained question: given how claims have developed at each age in the past, what development factor should we apply to claims currently at that age?

DeepTriangle asks a broader question:

Given the **whole development history observed so far**, together with patterns learned across many insurers and information about which insurer produced the triangle, what sequence of payments and case reserves should come next?

It is therefore still an **aggregate reserving model**. It does not model individual claims. Its basic observations are accident-year/development-year cells from development triangles. What changes is the way the development pattern is learned. ([arXiv](#))

Paper at a glance

Question	DeepTriangle
Dataset	Real insurance regulatory data from National Association of Insurance Commissioners Schedule P
Granularity	Aggregate accident-year × development-year triangle cells, not individual claims
Coverage	Accident years 1988–1997; 10 development years; 50 insurers in each of four lines of business
Dataset modality	Mixed structured data: multivariate sequential triangle data plus a categorical insurer identifier
Inputs	Incremental paid losses, case outstanding, net earned premium and insurer identifier
Code available	Yes
Language	R , using the Keras R interface and TensorFlow
Machine-learning technique	Gated recurrent unit encoder-decoder, sequence-to-sequence modelling, multi-task learning, company embeddings, dropout and an ensemble of neural networks
Learning paradigm	Supervised learning
Task	Multivariate regression and forecasting
Main reserving output	Forecast future incremental paid claims and accumulate these with payments already observed to estimate ultimate claims

Kuo provides code for the experiments. The paper states that DeepTriangle was implemented using the **Keras R package and TensorFlow**. ([arXiv](#))

1. Where DeepTriangle came from

It helps to begin with ordinary chain ladder.

Suppose $(C_{i,j})$ is cumulative paid claims for accident year (i) at development age (j) . In simplified form, chain ladder assumes that:

$$C_{i,j+1} \approx f_j C_{i,j}$$

where (f_j) is the development factor taking claims from development age (j) to $(j+1)$.

The important feature is that **the same (f_j) is applied across accident years**.

For example, if the estimated development factor from development year 3 to development year 4 is 1.20, then the model essentially says that accident years currently at development year 3 are expected to grow by approximately 20%, regardless of whether they arose in 1992, 1995 or 1997.

This does **not** quite mean that accident year and development year are statistically independent. That wording is too strong.

A better description is:

The development pattern is assumed to be homogeneous across accident years. There is no interaction allowing the development factor itself to change according to accident year.

That distinction becomes clearer in the Generalized Linear Model, or **GLM**, representation of chain ladder.

A common over-dispersed Poisson formulation for incremental claims has:

$$\log E[C_{ij}] = c + \alpha_i + \beta_j$$

where:

- (α_i) captures the accident-year effect;
- (β_j) captures the development-year effect;
- (c) is an overall level.

Notice what is missing:

$$\alpha_i \times \beta_j$$

There is no explicit accident-year/development-year interaction.

The accident-year and development-year effects are therefore **separable**.

That structure is one reason chain ladder is so robust and interpretable. It is also a limitation. If inflation, claims handling, legal developments or claim mix cause development patterns themselves to change over calendar time, a single development factor may no longer describe every accident year well.

DeepTriangle is ultimately trying to relax this type of restriction.

2. Wüthrich’s neural-network development-factor idea

Mario Wüthrich’s earlier neural-network work provides a useful bridge between traditional chain ladder and DeepTriangle.

Wüthrich starts much closer to chain ladder.

Instead of assuming one development factor:

$$f_j$$

for everybody at development age (j), he allows the factor to depend on claim characteristics:

$$f_j(x)$$

where (x) contains additional information about the claim.

His application uses individual claim data rather than only aggregate triangles. A feed-forward neural network learns how development behaviour varies according to characteristics of the underlying claims.

Conceptually:

Method	Development assumption
Ordinary chain ladder	One (f_j) at each development age
Wüthrich neural approach	(f_j(x)), allowing development to vary with claim characteristics
DeepTriangle	Does not explicitly estimate (f_j); learns the whole future development sequence from the history

This is an important progression.

Wüthrich is effectively saying:

“Keep the idea of development factors, but allow them to be heterogeneous.”

DeepTriangle goes further:

“Perhaps we do not need to explicitly specify development factors at all. Let the neural network learn the development dynamics from sequences.”

There is nevertheless a shared warning. Once development depends strongly on time, claim characteristics or other covariates, extrapolation becomes more difficult.

A neural network can model historical heterogeneity. It cannot automatically know that a pattern seen historically will continue ten years into the future.

3. What is Mack chain ladder?

Kuo compares DeepTriangle against the **Mack chain-ladder model**.

This can sound like a fundamentally different reserving method from ordinary chain ladder, but it is not.

The central reserve estimate produced under Mack is essentially the familiar chain-ladder estimate.

What Thomas Mack added was a **stochastic framework around chain ladder**.

Ordinary chain ladder might give an actuary an estimate such as:

Best estimate reserve = £100 million.

The immediate question is:

How uncertain is that £100 million?

Mack provides assumptions that allow the actuary to calculate quantities such as the **standard error** and **mean squared error of prediction** of the chain-ladder forecast.

So the simplest way to remember Mack is:

Chain ladder gives the central estimate. Mack puts error bars around the chain-ladder machinery.

Mack does this without requiring the actuary to specify a complete probability distribution for the claims.

This is useful because it preserves much of the traditional chain-ladder framework while providing a mathematically structured estimate of prediction uncertainty.

One important weakness remains. Mack still relies on the underlying chain-ladder development structure. If the development factors themselves cease to be representative because the portfolio has changed, putting a stochastic framework around those factors does not solve the structural problem.

4. What is the bootstrap over-dispersed Poisson model?

Another benchmark in Kuo's paper is the **bootstrap over-dispersed Poisson model**, commonly shortened to bootstrap ODP.

There are two separate ideas here.

First: the over-dispersed Poisson model

The over-dispersed Poisson model is a GLM representation of incremental claims.

A simplified expectation structure is:

$$\log(m_{ij}) = c + \alpha_i + \beta_j$$

where (m_{ij}) is expected incremental claims for accident year (i) and development year (j).

The variance is assumed to follow:

$$\text{Var}(C_{ij}) = \phi m_{ij}$$

The parameter (ϕ) is the **dispersion parameter**.

An ordinary Poisson distribution has:

$$\text{Var}(C) = E[C]$$

which is usually much too restrictive for insurance claims.

With an over-dispersed Poisson model:

$$\text{Var}(C) = \phi E[C]$$

so when $(\phi > 1)$, considerably more variability is allowed.

Under the standard chain-ladder formulation, the fitted means reproduce the familiar chain-ladder point estimates.

So over-dispersed Poisson is not mainly interesting because it produces a radically different best estimate. Its value is that it gives chain ladder a useful statistical model.

Second: the bootstrap

The bootstrap then asks:

What if the historical triangle we observed were only one possible realisation of the claims process?

After fitting the model, we examine its residuals — broadly, the differences between observed and fitted claims.

We repeatedly resample those residuals and use them to construct alternative plausible triangles.

For each simulated triangle, we refit the reserving model and calculate another reserve.

If we do this many times, instead of obtaining one number such as:

100

we might obtain something resembling:

91, 96, 102, 106, 113, ...

These simulations allow us to construct a distribution of possible reserve outcomes.

More sophisticated versions also add **process uncertainty** by simulating the future claims themselves after the parameters have been estimated.

That distinction is crucial when comparing bootstrap ODP with DeepTriangle.

DeepTriangle gives very good **point forecasts** in Kuo's experiment.

Bootstrap ODP gives something DeepTriangle does not provide in this paper:

a route towards an actuarial predictive distribution of reserve outcomes.

For capital modelling and reserve-risk work, that is a significant advantage.

5. The DeepTriangle dataset

Kuo uses **real data**, not simulated claims.

The data come from National Association of Insurance Commissioners Schedule P regulatory filings. They contain accident years 1988–1997 with development experience extending to ten development years. ([arXiv](#))

Four lines of business are used:

- commercial auto;
- private passenger auto;
- workers' compensation;
- other liability.

For each line of business, Kuo selects **50 insurers**. A separate DeepTriangle model is developed for each line of business. Within a particular line, data from all 50 insurers are used together. ([arXiv](#))

This is important.

DeepTriangle is **not one enormous model spanning every business line**.

Conceptually it is:

Commercial auto → one model trained across 50 insurers
Private passenger auto → one model trained across 50 insurers
Workers' compensation → one model trained across 50 insurers
Other liability → one model trained across 50 insurers

The data are aggregated at:

accident year × development year × insurer × line of business

rather than individual claim level.

The dataset modality is therefore best described as **mixed structured sequential data**.

The paid and outstanding triangles are sequential numerical information. The insurer code is categorical.

6. Paid, incurred and case outstanding

Kuo uses:

- cumulative paid claims;

- incurred claims;
- net earned premium;
- development lag;
- accident year;
- insurer code.

For DeepTriangle:

$\text{case outstanding} = \text{incurred} - \text{cumulative paid}$

or equivalently:

$$\text{incurred} = \text{cumulative paid} + \text{case outstanding}$$

So the “outstanding” quantity is essentially the insurer’s **case reserves** at that development date. ([arXiv](#))

DeepTriangle converts cumulative paid amounts into **incremental paid claims**.

It then scales both incremental paid and case outstanding by net earned premium.

Thus an observation can be represented roughly as:

$$Y_{i,j} = \left(\frac{P_{i,j}}{NPE_i}, \frac{OS_{i,j}}{NPE_i} \right)$$

where:

- $(P_{i,j})$ is incremental paid;
- $(OS_{i,j})$ is case outstanding;
- (NPE_i) is net earned premium.

Kuo explains that this scaling makes training easier because companies of very different sizes are put onto roughly comparable scales. ([arXiv](#))

7. The central DeepTriangle idea: treat development as a sequence

Suppose an accident year has currently been observed through three development periods:

$$Y_1, Y_2, Y_3$$

Traditional chain ladder pays particular attention to the current maturity and the relevant development factor.

DeepTriangle instead feeds the historical sequence:

$$(Y_1, Y_2, Y_3)$$

into the neural network and asks it to forecast:

$$(Y_4, Y_5, \dots, Y_{10})$$

Each (Y_j) contains both:

- incremental paid claims; and
- case outstanding.

This is a **sequence-to-sequence forecasting problem**.

The phrase “sequence-to-sequence” simply means:

sequence in → sequence out.

Language translation provides a familiar machine-learning example:

sequence of English words → sequence of French words.

DeepTriangle instead does:

observed claims-development sequence → future claims-development sequence.

This is substantially more flexible than estimating one scalar development factor at a time.

8. Why use a gated recurrent unit?

Kuo uses a **gated recurrent unit**, or GRU.

A GRU is a form of recurrent neural network designed for sequential information.

A useful analogy is an actuary working across an accident-year row while keeping a running notebook.

At each development period the actuary asks:

What from the earlier development history should I remember?

and:

How much should the latest observation change my view?

The GRU’s internal hidden state performs something roughly analogous to that notebook.

DeepTriangle uses an **encoder-decoder architecture**.

The encoder reads the observed claims history and compresses it into an internal representation.

The decoder then uses that representation to generate states corresponding to future development periods.

Kuo uses 128 hidden units in both the encoder and decoder and applies 20% dropout. ([arXiv](#))

The important actuarial consequence is that predictions can depend upon the **pattern of the entire development history**, not merely the latest cumulative claim value.

For example, two accident years could have exactly the same cumulative paid amount at development year 4 but have reached it through very different paths.

Chain ladder may treat them similarly.

DeepTriangle has the potential to distinguish them.

9. Multi-task learning: why predict case outstanding as well?

DeepTriangle simultaneously predicts:

1. future incremental paid claims; and
2. future case outstanding.

This is called **multi-task learning**.

Rather than training one neural network to do one job, related prediction tasks share part of the same model.

Kuo gives several reasons why this might help.

Case outstanding contains information about the underlying claims process that may help the network predict subsequent payments.

Trying to predict case reserves also provides another training signal. This may discourage the network from finding a peculiar set of weights that happens to fit paid claims very well but does not represent the broader development process.

Kuo therefore treats case outstanding as an **auxiliary output**. The ultimate quantity of primary interest remains paid claims. ([arXiv](#))

This is sensible, but there is a real production risk.

Case reserves reflect more than underlying claim severity. They also reflect:

- claims-handler judgement;
- company reserving philosophy;
- operational processes;
- settlement strategy;
- changes in case-reserving guidelines.

If those practices change, the historical relationship between case outstanding and eventual payments may also change.

DeepTriangle may therefore be very good at learning a particular reserving philosophy — which can become a weakness when that philosophy changes.

10. Company embeddings: are they a balancing item?

Each company has a categorical identifier.

Simply giving a neural network company codes such as:

$$1, 2, 3, \dots, 50$$

would incorrectly imply that Company 40 is somehow numerically “twice” Company 20.

Instead, Kuo uses an **embedding**.

Each insurer is assigned a learned vector containing 49 numbers. ([arXiv](#))

Conceptually:

$$\text{Company 337} \rightarrow [e_1, e_2, \dots, e_{49}]$$

Those numbers are learned during neural-network training.

Kuo suggests that the embedding might capture persistent characteristics such as company size or case-reserving philosophy. Companies whose claim-development behaviour is similar should acquire similar representations. ([arXiv](#))

There is some truth in thinking of the embedding as a **balancing item**, but that interpretation needs care.

It is tempting to imagine the model doing:

1. explain claims using paid and outstanding history;
2. calculate the remaining residual;
3. use company embedding to explain that residual.

That is **not literally what happens**.

The embedding is trained simultaneously with the rest of the neural network.

There is no explicit residual-fitting stage.

A better actuarial analogy is a highly flexible **company-specific effect**.

The sequential claims data explain whatever patterns they can. At the same time, the embedding gives the model a way of saying:

“Even after considering those development patterns, this happens to be Company A, which historically behaves differently from Company B.”

In that sense, your balancing-item intuition is useful.

But there is an important warning.

There are only 50 companies and the embedding has **49 dimensions**.

That is a very rich company representation. It is therefore not really aggressive dimensionality reduction.

There is a possibility that the network learns something quite close to company identity rather than a compact, genuinely transferable description of insurer behaviour.

And the experiment does not demonstrate that the learned embeddings transfer naturally to an insurer the model has never seen.

Kuo explicitly says that adding new companies requires enlarging the embedding input and refitting the network. ([arXiv](#))

11. What exactly are the 100 neural networks?

For **each line of business**, Kuo trains an ensemble of **100 separate neural networks**.

They have:

- the same architecture;
- the same prediction problem;
- broadly the same training data;

but different **random starting weights**. ([arXiv](#))

So imagine:

$$M_1, M_2, M_3, \dots, M_{100}$$

All 100 are genuinely trained.

This is **not** one trained network being run 100 times at inference.

Neural-network optimisation is stochastic. Starting from different random weights can cause otherwise identical networks to converge to slightly different solutions.

Kuo therefore obtains 100 forecasts:

$$\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{100}$$

and averages them:

$$\hat{y}_{\text{ensemble}} = \frac{1}{100} \sum_{m=1}^{100} \hat{y}_m$$

This reduces the variance caused by random neural-network optimisation. Kuo explicitly says this is the reason for the ensemble. ([arXiv](#))

So there are **two separate stages**.

Training stage

Train 100 independent models.

Inference stage

Run all 100 models and average their predictions.

Your intuition was therefore partly right: **the averaging happens at inference**.

But generating the ensemble requires 100 independent training runs beforehand.

With four lines of business, the experiment therefore involves:

$$4 \times 100 = 400$$

fitted DeepTriangle networks.

That is much more computationally expensive than the common production arrangement of training one neural network and deploying it.

Today we would usually describe this as a **deep ensemble**.

12. Are those 100 models measuring reserve uncertainty?

Not in the actuarial sense.

Kuo explicitly warns that the 100 models should **not** be interpreted as a predictive reserve distribution. ([arXiv](#))

What the ensemble primarily captures is sensitivity to neural-network optimisation.

Suppose the 100 predicted reserves are all very similar:

$$99, 100, 101, 100, \dots$$

That tells us the fitted answer is relatively insensitive to random initial weights.

Suppose instead they vary substantially:

$$75, 89, 102, 123, 140, \dots$$

That tells us the neural-network optimisation is unstable.

This is useful information, but it is only one narrow component of **model uncertainty**.

It does not tell us how random future claim payments themselves may be.

13. A simple way to extend DeepTriangle towards uncertainty

The simplest extension would be to retain all 100 network predictions rather than only reporting their mean.

You could calculate empirical quantiles such as:

$$Q_{0.05}, \quad Q_{0.50}, \quad Q_{0.95}$$

But I would **not** call the interval between ($Q_{\{0.05\}}$) and ($Q_{\{0.95\}}$) a 90% reserve confidence interval.

It would be better labelled:

variation across independently trained neural networks

because that is what it measures.

A second relatively simple technique would be **Monte Carlo dropout**.

Dropout randomly switches some neural-network units off during training. Normally dropout is disabled when producing forecasts.

With Monte Carlo dropout, you leave this randomness active during inference and produce many forecasts from the same fitted model.

Again, the spread provides some information about model uncertainty.

For a more complete actuarial reserve distribution, however, I would want at least three components:

1. **parameter/model uncertainty**, such as differences across fitted neural networks;
2. **data uncertainty**, perhaps through bootstrap resampling of historical triangles;
3. **future process uncertainty**, representing randomness in future claims themselves.

This begins to look much more like what bootstrap ODP already provides.

For example, one could bootstrap historical triangles and refit DeepTriangle to each bootstrap sample.

The computational cost becomes substantial.

If we use:

100

bootstrap samples and train:

20

neural networks for each, that already means:

$$100 \times 20 = 2,000$$

neural-network training runs.

A more modern alternative would be to modify the output layer so that DeepTriangle predicts an entire probability distribution, or selected quantiles, rather than just a conditional mean.

But that would be a new model rather than what Kuo actually implements.

14. Training, validation and testing: an important distinction

There are three different data subsets involved, and they are easy to conflate.

Training information

Kuo states:

data as of year-end 1997 is used for training.

So imagine that the actuary is standing on **31 December 1997**.

The model receives the development triangle that would have been available on that date. (arXiv)

This is not simply “train on 1988–1995.”

Validation data

Neural networks can overfit if training continues too long.

Kuo therefore uses **early stopping**.

Each network can train for up to 1,000 epochs. If validation loss fails to improve over a 200-epoch window, training stops and the model returns to the weights from the best validation epoch. (arXiv)

The validation subset consists of observations within the training data that **became available after calendar year 1995**. (arXiv)

So 1995 is mainly relevant to the **validation split used during model fitting**.

It is not the cut-off for the final out-of-time test.

Final out-of-time test

The true test is much more interesting.

At year-end 1997, different accident years have reached different levels of maturity.

For example:

Accident year	Development observed by year-end 1997	Remaining periods to development 10
1988	10	0
1989	9	1
1990	8	2
1991	7	3
1992	6	4
1993	5	5
1994	4	6
1995	3	7
1996	2	8
1997	1	9

So for accident year 1989 the model essentially needs to forecast only one additional development period.

For accident year 1997 it needs to forecast **nine**.

Development year 10 for accident year 1997 would only emerge around calendar year 2006.

Kuo then compares the model's predictions with the subsequently observed cumulative paid loss at development year 10. ([arXiv](#))

So this is **not a two-year forecast from 1995 to 1997**.

It is an out-of-time triangle-completion exercise with projection horizons ranging from zero to nine development periods.

15. Why forecast horizon matters

Your intuition that errors should generally become more serious as the projection horizon increases is sound.

Suppose one accident year is already nine years developed.

DeepTriangle only needs to predict:

$$Y_{10}$$

There is relatively little remaining development.

Now consider the newest accident year.

The model may need to predict:

$$Y_2, Y_3, \dots, Y_{10}$$

Every predicted period creates another opportunity to deviate from reality.

A recurrent model may also compound errors indirectly because it is trying to infer a much longer future trajectory from a much shorter observed history.

The important correction is that **the later accident years are less mature, not more mature**.

Accident year 1997 has only one year of development available at the year-end 1997 valuation date.

So the hardest problem is usually:

newest accident year + longest remaining development horizon.

The paper's headline error statistics do not give us a clean plot of error against forecast horizon.

That would have been extremely useful.

For example, I would like to see something like:

$$\text{MAPE}(h)$$

where (h) is the number of future development periods being predicted.

That would tell us whether DeepTriangle's advantage over chain ladder holds at one-year, three-year and nine-year horizons separately.

16. How tolerant is DeepTriangle of delayed company data?

This issue is not directly tested in the paper, but it matters considerably for production deployment.

The architecture has one useful property: it is trained using histories of **different lengths**.

For one training example it might receive:

$$Y_1, Y_2$$

and predict the remainder.

For another it might receive:

$$Y_1, \dots, Y_7$$

and predict only the last few periods.

Therefore the model is naturally capable of forecasting from different levels of maturity.

That gives it some tolerance to missing recent development.

But a reporting delay still has a cost.

Suppose an accident year should currently have four years of observed development:

$$Y_1, Y_2, Y_3, Y_4$$

but the latest company submission is delayed, leaving only:

$$Y_1, Y_2, Y_3$$

The model can still forecast.

But instead of forecasting:

$$Y_5, \dots, Y_{10}$$

it now effectively has to forecast:

$$Y_4, Y_5, \dots, Y_{10}$$

The missing information has increased the forecast horizon by one period.

This matters much more for immature accident years.

Losing one observation from a nine-year-developed accident year is usually less serious than losing one observation when only two observations existed in the first place.

17. The bigger operational problem: inconsistent valuation dates

There is another issue when pooling data from 50 companies.

Suppose:

- Company A has reported through December;
- Company B has reported through September;
- Company C has revised its June numbers;
- Company D has not yet supplied its latest triangle.

You no longer have a clean common valuation date.

That creates a risk of **look-ahead bias**.

For example, the model might indirectly learn from December information from Company A while trying to forecast a Company B position that supposedly represents September.

A robust production implementation would therefore need a strict **as-of-date rule**:

Every item used in the model must genuinely have been available by the valuation date being modelled.

This is similar to the discipline required when building financial time-series models.

Using information that was published later but retrospectively assigned to an earlier date can produce excellent historical results and disastrous live performance.

18. Would an insurer really have data from 50 competitors?

Probably not at the frequency required for a live reserving process.

Kuo recognises this problem.

The Schedule P dataset is convenient for research because the regulatory filings provide comparable insurer-level information.

A real insurer will generally not know competitors' current internal case reserves and payment triangles before they are publicly filed.

A more realistic deployment would therefore be within an insurance group.

For example:

20 legal entities × common claims system × common quarterly valuation process.

DeepTriangle could learn across those related entities.

Another possibility would be modelling portfolios rather than legal companies:

motor bodily injury motor property damage commercial liability regional portfolios

The company-embedding mechanism could then become a **portfolio embedding**.

That may actually be more useful operationally than trying to reproduce the exact 50-company research setup.

19. Evaluation against other methods

Kuo compares DeepTriangle with several alternatives:

- Mack chain ladder;
- bootstrap over-dispersed Poisson;
- Bayesian reserving models;
- an automated machine-learning benchmark.

The automated machine-learning benchmark searches across methods including random forests, extremely randomised forests, gradient boosting machines, feed-forward neural networks and stacked ensembles. ([arXiv](#))

Kuo evaluates ultimate-loss prediction using two measures.

The first is **mean absolute percentage error**, or MAPE.

Very roughly:

$$MAPE \approx \frac{1}{n} \sum_{i=1}^n \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

It measures the average absolute error as a percentage of the actual outcome.

The second is **root mean squared percentage error**, or RMSPE:

$$RMSPE \approx \sqrt{\frac{1}{n} \sum_{i=1}^n \left(\frac{\hat{y}_i - y_i}{y_i} \right)^2}$$

Because errors are squared before averaging, RMSPE penalises large misses more heavily than MAPE.

20. Results

DeepTriangle achieves the lowest reported MAPE and RMSPE across all four lines of business. ([arXiv](#))

For MAPE:

Line of business	Mack	DeepTriangle
Commercial auto	6.0%	4.3%
Other liability	13.4%	10.9%
Private passenger auto	3.8%	2.5%
Workers' compensation	5.3%	4.6%

For RMSPE:

Line of business	Mack	DeepTriangle
Commercial auto	8.0%	5.7%
Other liability	20.2%	15.0%
Private passenger auto	6.1%	3.9%
Workers' compensation	7.9%	6.7%

The improvement is meaningful, particularly for private passenger auto and commercial auto.

But the conclusion should not be:

“Deep learning beats chain ladder.”

The defensible conclusion is much narrower:

On this particular Schedule P experiment, using these four lines, this model architecture and these evaluation measures, DeepTriangle produced better out-of-time ultimate-loss point forecasts than the benchmark methods tested.

That is still an interesting result.

21. An important failure case

Kuo also examines individual company development patterns rather than only aggregated error statistics.

For Company 1767’s commercial auto business, DeepTriangle captures the observed development relatively well.

For Company 337’s workers’ compensation business, however, the model fails to anticipate deteriorating loss ratios. ([arXiv](#))

This is one of the most useful observations in the paper.

DeepTriangle can learn complicated relationships from historical development.

But suppose something genuinely changes:

- inflation accelerates;
- court awards rise;
- claims handling deteriorates;
- claims mix changes;
- legislation changes;
- latent claims emerge.

If the information available before the change does not contain a sufficient signal, the neural network cannot magically forecast the regime shift.

This is the same fundamental problem that affects traditional reserving models, although flexible models may sometimes hide it better because their historical fit looks more impressive.

22. What DeepTriangle really contributes

The most important contribution is not the use of a GRU by itself.

It is the change in modelling philosophy.

Traditional chain ladder is roughly:

current maturity + development factor → future claims

Wüthrich extends this towards:

current claims + claim characteristics → heterogeneous development factor

DeepTriangle moves towards:

whole observed development sequence + case-reserve behaviour + insurer identity → whole future development sequence

That is a substantial increase in flexibility.

It allows information to be shared across companies while retaining company-specific behaviour.

It also naturally uses paid claims and case outstanding together rather than treating them as entirely separate analyses.

And because the model works directly with sequential development histories, it can learn relationships that would require explicitly specified interaction terms in a GLM.

23. What DeepTriangle does not solve

The paper also leaves several major problems open.

Reserve uncertainty

DeepTriangle provides point estimates. The 100-network ensemble is not a calibrated reserve distribution.

Structural breaks

The model can fail when future development departs from patterns represented in historical data.

Explainability

Chain-ladder development factors can be inspected directly.

A 128-unit GRU combined with a 49-dimensional company embedding and fully connected layers is considerably harder to explain.

New companies

A completely new company does not arrive with a meaningful learned embedding. The model must be modified and refitted. ([arXiv](#))

Data availability

The research experiment benefits from a large, harmonised regulatory dataset. Real insurers may not have timely equivalent data from external companies.

Long-horizon extrapolation

Recent accident years require the longest forecasts and have the least observed information. That is exactly where reserving uncertainty is generally greatest.

24. Overall assessment

DeepTriangle is interesting because it does **not** discard the actuarial triangle.

Instead, it asks whether the triangle can be treated as a richer machine-learning object.

The answer appears to be yes.

The development row becomes a sequence.

Paid and outstanding claims become related prediction tasks.

Company identity becomes a learned representation.

Several insurers contribute information to the same model.

And rather than specifying development factors explicitly, the network learns a nonlinear mapping from observed development history to future development.

The strongest feature of the paper is therefore not simply “deep learning for reserving.”

It is the demonstration that **aggregate reserving can be reframed as supervised multivariate sequence forecasting**.

That gives actuaries a useful middle ground between traditional aggregate reserving and full individual-claims modelling.

But it should not be viewed as a complete replacement for classical stochastic reserving.

DeepTriangle is strongest where the objective is **point prediction**.

Mack and bootstrap ODP remain much more developed when the question becomes:

“How uncertain is this reserve?”

And the Company 337 example is a useful reminder that increasing model sophistication does not remove the fundamental difficulty of forecasting changes that are not represented in the historical data.

Five-sentence recap

DeepTriangle is a **supervised aggregate reserving model** that uses real Schedule P development triangles rather than individual claims. It uses a gated recurrent unit sequence-to-sequence architecture to forecast future incremental paid claims and case outstanding jointly, while learned insurer embeddings allow persistent differences between companies to influence the forecast. Kuo fits **100 independently initialised neural networks for each line of business and averages their predictions**, which stabilises the point forecast but does not constitute a reserve uncertainty distribution. The final evaluation is not simply a 1995–1997 holdout: the model uses information available at year-end 1997 and is tested against subsequent development through development year 10, meaning the newest accident year requires roughly nine future development periods to be forecast. DeepTriangle beats Mack chain ladder, bootstrap ODP and the other tested benchmarks on point-prediction error in this experiment, but uncertainty estimation, structural breaks, long-horizon extrapolation and production data availability remain important limitations.

Concepts a reader should now be able to explain

- Why ordinary chain ladder assumes broadly homogeneous development patterns across accident years.
- Why the GLM representation has separable accident-year and development-year effects rather than an interaction between them.
- How Wüthrich allows development factors to depend on additional claim characteristics.
- Why Mack chain ladder is principally a stochastic framework around familiar chain-ladder estimates.
- How bootstrap ODP creates a distribution of possible reserve outcomes.
- Why DeepTriangle remains aggregate reserving despite using deep learning.
- Why a GRU is suitable for development histories.
- What sequence-to-sequence forecasting means in a reserving context.
- Why paid claims and case outstanding are predicted together.
- Why company embeddings resemble flexible company-specific effects but are not literally fitted to residuals in a second stage.
- Why Kuo trains 100 separate neural networks for each line of business.
- Why the spread of those 100 predictions is not the same as a full reserve uncertainty distribution.

- Why the 1997 accident year is much harder to forecast than the 1989 accident year at the year-end 1997 valuation date.
- Why delayed company submissions effectively lengthen the forecast horizon.
- Why strict valuation-date controls would be essential in a live multi-company implementation.